

ML Math Roadmap: What to Study and How Much

By Nilesh Hadalgi (@techie_programmer)

How to Use This Roadmap

Machine learning looks intimidating because the math seems endless. It isn't. Almost everything you meet in ML comes from five areas, and you don't need a math degree to be productive in any of them. You need the right topics, in the right order, with the right amount of time on each.

This roadmap tells you exactly what to study in each area, why it matters for ML, how to know you've learned it, and where to learn it from.

Three rules before you start:

- **Code every concept.** After each topic, implement it in NumPy or Python. If you can't code it, you haven't learned it.
- **Learn just in time.** You don't have to finish all the math before touching ML. Alternate between math and models.
- **Aim for intuition first, proofs second.** You should be able to explain each idea in plain words and draw it, before you worry about formal proofs.

The Split at a Glance

Area	Share of study time	What it gives you in ML
Linear Algebra	35%	The language of data, models and weights
Probability and Statistics	25%	How models handle uncertainty and evaluate results
Multivariate Calculus	20%	How models learn through gradients
Optimization	10%	How training actually finds good parameters

Area	Share of study time	What it gives you in ML
Information Theory	10%	Where loss functions and divergence measures come from

These percentages are a guide for how to divide your study hours, not a rule about how much math appears in any single paper.

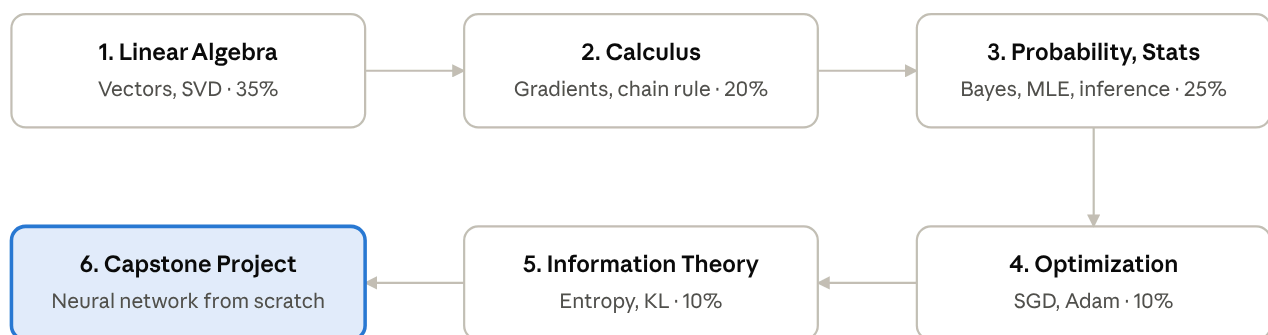
Recommended Order

The areas depend on each other, so the order matters:

1. **Linear algebra** first, because everything else is written in vectors and matrices.
2. **Multivariate calculus** second, because gradients are vectors and Jacobians are matrices.
3. **Probability and statistics** third, because it builds on both and is where ML starts to feel like ML.
4. **Optimization** fourth, because it combines calculus and linear algebra to explain training.
5. **Information theory** last, because it uses probability and gives meaning to loss functions.

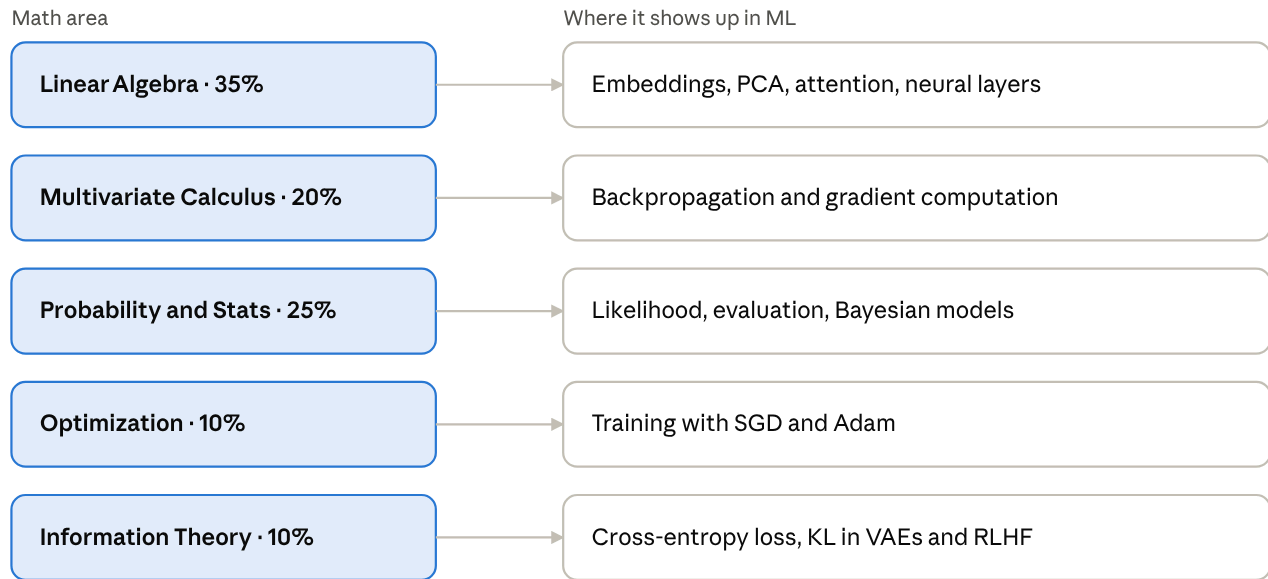
In practice you will overlap these, but this is the backbone.

Learn the math in this order, then build a capstone



study path · 6 steps

Study in the numbered order; the percentages show how to split your hours.

Each math area powers a different part of the ML workflow

math areas mapped to ML uses · 5 rows

Use this map to find which area to revisit when an ML concept feels unclear.

1. Linear Algebra (35%)

Linear algebra is the single biggest chunk because data, images, text embeddings and model weights are all vectors, matrices or tensors. A neural network layer is a matrix multiplication plus a nonlinearity.

Topics

Vectors and spaces

- Vectors, vector addition, scalar multiplication
- Dot product, norms (L1, L2, L-infinity), distance and angle between vectors
- Linear combinations, span, linear independence
- Basis, dimension, subspaces

Matrices and linear transformations

- Matrix as a transformation of space
- Matrix multiplication, transpose, inverse, identity
- Rank, null space, column space

- Determinants (the intuition: how a transformation scales volume)
- Solving linear systems, Gaussian elimination

Decompositions (the most important part for ML)

- Eigenvalues and eigenvectors
- Diagonalization and symmetric matrices
- Singular Value Decomposition (SVD)
- Orthogonality, projections, Gram-Schmidt, QR decomposition
- Positive definite matrices

Tensors and practical skills

- Tensors as generalizations of matrices
- Broadcasting and batched operations in NumPy and PyTorch

Where it shows up in ML

- Linear and logistic regression (the normal equation, least squares)
- PCA (eigenvectors of the covariance matrix, or SVD)
- Embeddings and similarity search (dot product, cosine similarity)
- Neural network layers and attention (matrix multiplications)
- Recommender systems (matrix factorization)

Milestones

- You can explain what an eigenvector is using a picture, not a formula.
- You can implement PCA from scratch using only NumPy.
- You can compute the shapes of every matrix in a small neural network forward pass without running code.

Practice projects

- Implement matrix multiplication, then compare with NumPy for speed.
- Compress an image with SVD and plot quality against number of singular values.
- Build a simple movie recommender with matrix factorization.

2. Probability and Statistics (25%)

ML models are built on uncertain data. Probability gives you the language to describe that uncertainty, and statistics gives you the tools to learn from data and judge whether a result can be trusted.

Topics

Probability foundations

- Sample spaces, events, axioms of probability
- Conditional probability, independence
- Bayes' theorem (spend extra time here)
- Law of total probability, chain rule of probability

Random variables and distributions

- Discrete and continuous random variables
- PMF, PDF, CDF
- Expectation, variance, covariance, correlation
- Key distributions: Bernoulli, Binomial, Poisson, Uniform, Gaussian (univariate and multivariate), Exponential, Beta, Dirichlet

Limit theorems

- Law of large numbers
- Central limit theorem

Statistical inference

- Maximum likelihood estimation (MLE)
- Maximum a posteriori estimation (MAP) and Bayesian inference
- Priors, posteriors, conjugacy
- Bias and variance of estimators
- Confidence intervals and hypothesis testing
- p-values and their limitations

Applied statistics for ML

- Bias-variance tradeoff
- Overfitting, cross-validation

- Evaluation metrics: accuracy, precision, recall, F1, ROC and AUC
- Sampling methods, bootstrapping
- Markov chains and basics of Monte Carlo methods

Where it shows up in ML

- Naive Bayes, Gaussian mixture models, hidden Markov models
- Loss functions as negative log-likelihood
- Regularization as a prior (L2 is a Gaussian prior, L1 is a Laplace prior)
- Generative models, VAEs, diffusion models
- A/B testing and model evaluation

Milestones

- You can derive linear regression and logistic regression loss from maximum likelihood.
- You can explain why test accuracy alone can be misleading.
- You can solve a Bayes' theorem problem without looking up the formula.

Practice projects

- Build a Naive Bayes spam classifier from scratch.
- Simulate the central limit theorem with different distributions and plot it.
- Fit a Gaussian mixture model with the EM algorithm using NumPy.

3. Multivariate Calculus (20%)

Calculus explains how a model learns. Training is the process of nudging parameters in the direction that reduces error, and that direction is a gradient.

Topics

Single-variable refresher

- Limits, derivatives, differentiation rules
- Chain rule (this is the one that matters most)
- Integrals at a basic level

Multivariable calculus

- Functions of several variables
- Partial derivatives
- Gradient vector and its geometric meaning
- Directional derivatives
- Jacobian matrix
- Hessian matrix and second-order behavior
- Multivariable chain rule
- Taylor series and local approximations

Matrix calculus

- Derivatives with respect to vectors and matrices
- Common identities, for example the gradient of a squared error loss

Integration for probability

- Multiple integrals, enough to understand expectations and marginalization

Automatic differentiation

- Computational graphs
- Forward mode and reverse mode (backpropagation)

Where it shows up in ML

- Backpropagation is the chain rule applied across a computational graph
- Gradient descent and every optimizer built on it
- Second-order methods using the Hessian
- Sensitivity analysis and saliency maps

Milestones

- You can derive the gradient of mean squared error for linear regression by hand.
- You can explain backpropagation on a two-layer network with the chain rule.
- You can implement a tiny autograd engine, like a minimal version of PyTorch's autograd.

Practice projects

- Write gradient descent for linear regression without any ML library.

- Implement backpropagation for a small MLP in NumPy and verify it with numerical gradient checking.
- Build a micro autograd library supporting add, multiply and tanh.

4. Optimization (10%)

Optimization connects calculus and linear algebra to the practical question of how to actually train a model.

Topics

- Objective functions, constraints, local and global minima
- Convex sets and convex functions, why convexity makes life easy
- Gradient descent, learning rate, convergence behavior
- Stochastic gradient descent and mini-batches
- Momentum, RMSProp, Adam and AdamW
- Learning rate schedules and warmup
- Saddle points, plateaus and why deep learning is non-convex
- Newton's method and the idea behind quasi-Newton methods (L-BFGS)
- Lagrange multipliers and constrained optimization
- Regularization (L1, L2) as constrained or penalized optimization
- Basic duality (useful for understanding SVMs)

Where it shows up in ML

- Training every neural network
- Support vector machines
- Hyperparameter tuning
- Fine-tuning large models

Milestones

- You can explain why Adam often works better than plain SGD, and when it may not.
- You can diagnose a training run where loss explodes or stalls using what you know about learning rates.

Practice projects

- Implement SGD, momentum and Adam from scratch and compare their paths on a 2D loss surface.
- Visualize loss landscapes for convex and non-convex functions.

5. Information Theory (10%)

Information theory is a small area with a big payoff, because it explains why the loss functions you use are the right ones.

Topics

- Information content and surprise
- Entropy
- Joint and conditional entropy
- Cross-entropy
- KL divergence (and why it is not symmetric)
- Mutual information
- Jensen-Shannon divergence
- The relationship between entropy, likelihood and compression
- Basic ideas of the information bottleneck

Where it shows up in ML

- Cross-entropy loss for classification and language models
- KL divergence in VAEs, distillation and reinforcement learning (PPO, RLHF)
- Decision trees choosing splits with information gain
- Perplexity as a language model metric

Milestones

- You can show that minimizing cross-entropy is the same as minimizing KL divergence to the data distribution.
- You can explain what a perplexity of 20 means.

Practice projects

- Compute entropy of text at character level for different languages.
- Implement a decision tree using information gain.

12-Week Study Plan

This plan assumes about 6 to 8 hours per week. Adjust the pace to your schedule.

Weeks	Focus	Output
1-3	Linear algebra: vectors, matrices, transformations, eigen, SVD	PCA from scratch
4-5	Multivariate calculus: gradients, Jacobians, chain rule	Backprop for a small MLP
6-8	Probability and statistics: Bayes, distributions, MLE, inference	Naive Bayes classifier
9-10	Optimization: SGD variants, convexity, regularization	Optimizer comparison notebook
11	Information theory: entropy, cross-entropy, KL	Decision tree with information gain
12	Integration: tie everything together	Capstone project

Capstone Projects

Pick one to prove you have connected the pieces:

- **Neural network from scratch:** Build, train and evaluate a classifier on MNIST using only NumPy. It uses linear algebra, calculus, probability, optimization and cross-entropy.
- **Mini autograd library:** Build a scalar autograd engine and train a small network with it.
- **Probabilistic classifier:** Build a Gaussian mixture model or Naive Bayes classifier and evaluate it with proper statistics.

Resources

Linear algebra

- 3Blue1Brown, Essence of Linear Algebra (YouTube) for intuition
- Gilbert Strang, MIT 18.06 Linear Algebra lectures and textbook

Probability and statistics

- Joe Blitzstein, Harvard Stat 110 lectures
- StatQuest with Josh Starmer (YouTube) for clear explanations of statistics for ML

Calculus

- 3Blue1Brown, Essence of Calculus
- Khan Academy multivariable calculus
- Andrej Karpathy, micrograd and his neural network lectures for backprop intuition

Optimization

- Boyd and Vandenberghe, Convex Optimization (free from the authors)
- Sebastian Ruder's overview of gradient descent optimization algorithms

Information theory

- Chris Olah's visual information theory post
- David MacKay, Information Theory, Inference, and Learning Algorithms (free from the author)

All-in-one books

- Deisenroth, Faisal and Ong, Mathematics for Machine Learning (free online)
- Goodfellow, Bengio and Courville, Deep Learning, Part I (applied math and ML basics)

Common Mistakes to Avoid

- **Studying only theory.** Without code, concepts fade quickly.
- **Skipping linear algebra intuition.** Memorizing formulas for eigenvalues without understanding transformations will hurt you later.
- **Trying to finish all the math first.** Learn enough, build something, then return to fill gaps.
- **Ignoring statistics.** Many people can train a model but cannot tell if the result is meaningful.
- **Collecting resources instead of studying.** Pick one main resource per topic and finish it.

Self-Check Before Moving On

You are ready for serious ML work when you can answer these without notes:

- What does a matrix do to a vector, geometrically?
- Why does PCA use eigenvectors of the covariance matrix?
- What is the gradient, and why does it point toward steepest increase?
- How does the chain rule produce backpropagation?
- Why is cross-entropy the natural loss for classification?
- What is the difference between MLE and MAP?
- Why does a bigger learning rate sometimes make training diverge?

If any of these feel shaky, revisit that section and rebuild the concept in code.